

Zeal College of Engineering and Research

Subject: LP-V

Name: Jenil Girish Rathod

Class: BE

Div: B

Batch: B1

Roll No: B22009

Group A: Practical No:1

Program Statement: Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS.

Implemented Code:

```
#include <iostream>
#include <omp.h>

using namespace std;

#define MAX 20

int graph[MAX][MAX];
int visited[MAX];
int n;

// ===== PARALLEL BFS =====
void parallelBFS(int start) {
    int frontier[MAX], next[MAX];
    int frontSize = 0, nextSize = 0;

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    frontier[frontSize++] = start;
    visited[start] = 1;

    cout << "\nParallel BFS: ";

    while (frontSize > 0) {
        nextSize = 0;

        #pragma omp parallel for
        for (int i = 0; i < frontSize; i++) {
            int node = frontier[i];
```

```

        #pragma omp critical
        cout << node << " ";

        for (int j = 0; j < n; j++) {
            if (graph[node][j] && !visited[j]) {
                #pragma omp critical
                {
                    if (!visited[j]) {
                        visited[j] = 1;
                        next[nextSize++] = j;
                    }
                }
            }
        }

        frontSize = nextSize;
        for (int i = 0; i < nextSize; i++)
            frontier[i] = next[i];
    }
}

// ===== PARALLEL DFS =====
void parallelDFSUtil(int node) {
    int processNode = 0;

    #pragma omp critical
    {
        if (!visited[node]) {
            visited[node] = 1;
            processNode = 1;
            cout << node << " ";
        }
    }

    if (!processNode) return;

    for (int i = 0; i < n; i++) {
        if (graph[node][i]) {
            #pragma omp task
            parallelDFSUtil(i);
        }
    }

    #pragma omp taskwait
}

void parallelDFS(int start) {
    for (int i = 0; i < n; i++)

```

```

        visited[i] = 0;

    cout << "\nParallel DFS: ";

    #pragma omp parallel
    {
        #pragma omp single
        parallelDFSUtil(start);
    }
}

// ===== MAIN =====
int main() {
    int edges, u, v, start;

    cout << "Enter number of nodes: ";
    cin >> n;

    // Initialize graph
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            graph[i][j] = 0;

    cout << "Enter number of edges: ";
    cin >> edges;

    cout << "Enter edges (u v) for undirected graph:\n";
    for (int i = 0; i < edges; i++) {
        cin >> u >> v;
        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    cout << "Enter starting node: ";
    cin >> start;

    parallelBFS(start);
    parallelDFS(start);

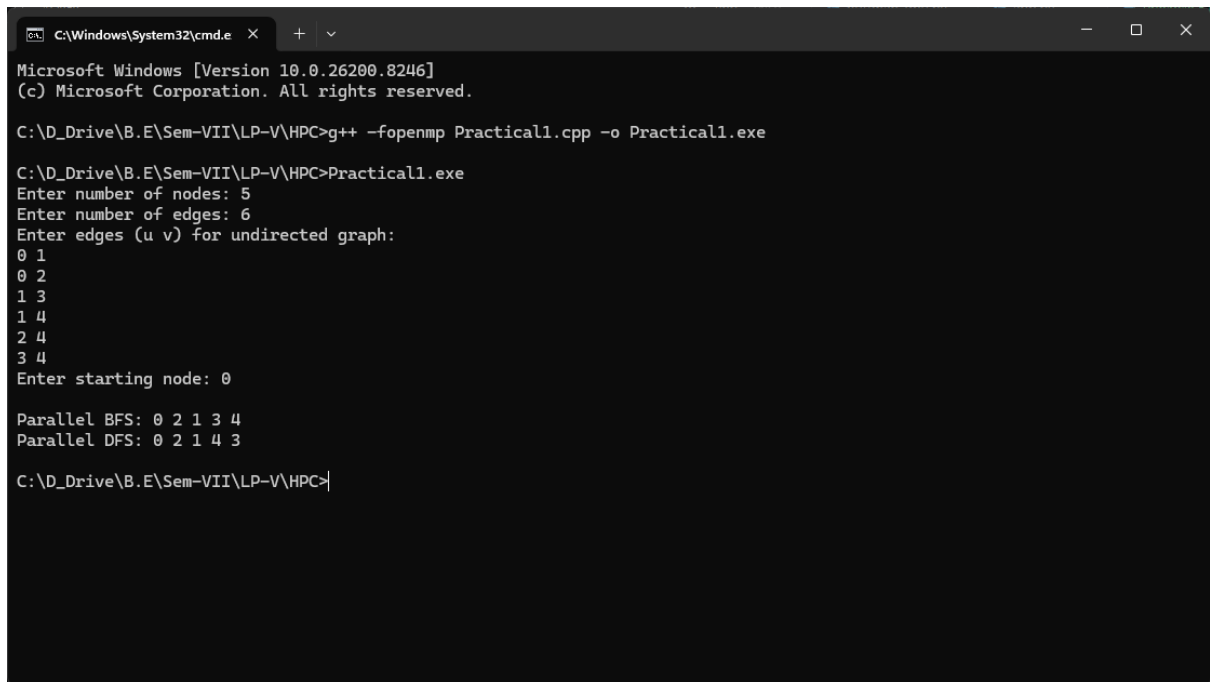
    cout << endl;
    return 0;
}

```

How to compile: g++ -fopenmp Practical1.cpp -o program

How to run: program.exe

OUTPUT:



```
C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\D_Drive\B.E\Sem-VII\LP-V\HPC>g++ -fopenmp Practicall.cpp -o Practicall.exe

C:\D_Drive\B.E\Sem-VII\LP-V\HPC>Practicall.exe
Enter number of nodes: 5
Enter number of edges: 6
Enter edges (u v) for undirected graph:
0 1
0 2
1 3
1 4
2 4
3 4
Enter starting node: 0

Parallel BFS: 0 2 1 3 4
Parallel DFS: 0 2 1 4 3

C:\D_Drive\B.E\Sem-VII\LP-V\HPC>
```